

**Laporan Mini Project**  
**Lampu Berjalan, Kontrol LDR, Dimming, dan WiFi Scanner**



**Mata Kuliah**  
Sistem Mikroprosesor dan Mikrokontroller

**Dosen Pengampu**  
Eko Pramunanto, S.T., M.T.

**Disusun oleh**  
Yuke Brilliant Hestiavin  
5024241016

Departemen Teknik Komputer  
Fakultas Teknologi Elektro dan Informatika Cerdas  
Institut Teknologi Sepuluh Nopember  
Surabaya, 3 Juni 2026

## Daftar Isi

|          |                                                                             |          |
|----------|-----------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Pendahuluan</b>                                                          | <b>1</b> |
| 1.1      | Latar Belakang . . . . .                                                    | 1        |
| 1.2      | Tujuan . . . . .                                                            | 1        |
| <b>2</b> | <b>Dasar Teori</b>                                                          | <b>1</b> |
| 2.1      | Pulse Width Modulation (PWM) . . . . .                                      | 1        |
| 2.2      | Analog to Digital Converter (ADC) . . . . .                                 | 2        |
| 2.3      | Sensor LDR . . . . .                                                        | 2        |
| 2.4      | Potensiometer . . . . .                                                     | 2        |
| 2.5      | WiFi Scanner Asinkron pada ESP32 . . . . .                                  | 2        |
| <b>3</b> | <b>Implementasi: Lampu Berjalan, Kontrol LDR, Dimming, dan WiFi Scanner</b> | <b>3</b> |
| 3.1      | Deskripsi . . . . .                                                         | 3        |
| 3.2      | Komponen yang Digunakan . . . . .                                           | 3        |
| 3.3      | Rangkaian (Wiring Diagram) . . . . .                                        | 3        |
| 3.4      | Kode Program . . . . .                                                      | 5        |
| 3.5      | Hasil dan Pembahasan . . . . .                                              | 8        |
| <b>4</b> | <b>Kesimpulan</b>                                                           | <b>9</b> |

# **1 Pendahuluan**

## **1.1 Latar Belakang**

Sistem pencahayaan otomatis dan kontrol nirkabel merupakan salah satu contoh penerapan sistem embedded dalam kehidupan sehari-hari. Pada sistem tersebut, mikrokontroler digunakan untuk membaca input dari sensor, memproses data, dan menghasilkan output berupa kendali aktuator.

Mini project ini menggabungkan beberapa konsep utama dalam mata kuliah Sistem Mikroprosesor dan Mikrokontroler, yaitu Pulse Width Modulation (PWM), Analog to Digital Converter (ADC), sensor cahaya LDR, pembacaan potensiometer, dan konektivitas WiFi pada ESP32. Sistem yang dibuat berupa lampu berjalan atau efek Knight Rider dengan pengaturan kecepatan dan kecerahan, kontrol otomatis berbasis LDR, serta fitur pemindaian jaringan WiFi secara asinkron.

## **1.2 Tujuan**

Tujuan dari mini project ini adalah sebagai berikut:

- Menerapkan teknik PWM untuk mengontrol kecerahan LED.
- Menggunakan potensiometer sebagai input analog untuk mengatur brightness dan kecepatan animasi lampu.
- Mengimplementasikan sensor LDR sebagai kontrol otomatis berdasarkan intensitas cahaya lingkungan.
- Memahami penggunaan beberapa kanal ADC pada ESP32 untuk membaca beberapa input analog secara bersamaan.
- Mengimplementasikan fitur WiFi scanning secara asinkron atau non-blocking pada ESP32.
- Menampilkan hasil pembacaan sensor, potensiometer, dan WiFi scanner melalui Serial Monitor.

# **2 Dasar Teori**

## **2.1 Pulse Width Modulation (PWM)**

Pulse Width Modulation atau PWM adalah teknik untuk mengatur daya rata-rata yang diberikan kepada beban dengan cara mengubah perbandingan waktu aktif dan tidak aktif dari sinyal digital. Perbandingan waktu aktif terhadap satu periode sinyal disebut duty cycle.

Pada LED, duty cycle yang lebih besar akan menghasilkan nyala yang lebih terang, sedangkan duty cycle yang lebih kecil akan menghasilkan nyala yang lebih redup. Dengan teknik PWM, ESP32 dapat mengontrol kecerahan LED tanpa harus menggunakan rangkaian analog tambahan.

## **2.2 Analog to Digital Converter (ADC)**

Analog to Digital Converter atau ADC digunakan untuk mengubah sinyal analog menjadi nilai digital. ESP32 memiliki fitur ADC yang dapat membaca tegangan analog pada pin tertentu. Nilai ADC pada ESP32 umumnya berada pada rentang 0 hingga 4095 karena resolusinya 12-bit.

Pada mini project ini, ADC digunakan untuk membaca dua buah potensiometer dan satu sensor LDR. Potensiometer pertama digunakan untuk mengatur kecepatan perpindahan LED, sedangkan potensiometer kedua digunakan untuk mengatur brightness LED. Sensor LDR digunakan untuk membaca kondisi intensitas cahaya lingkungan.

## **2.3 Sensor LDR**

Light Dependent Resistor atau LDR adalah sensor cahaya yang nilai resistansinya berubah sesuai dengan intensitas cahaya yang diterima. Ketika cahaya yang diterima semakin terang, resistansi LDR akan berubah sehingga tegangan pada rangkaian pembagi tegangan juga berubah.

Nilai tegangan tersebut dapat dibaca oleh pin ADC ESP32. Dengan membandingkan nilai ADC terhadap nilai ambang batas atau threshold, sistem dapat menentukan apakah kondisi lingkungan sedang terang atau gelap.

## **2.4 Potensiometer**

Potensiometer adalah komponen resistor variabel yang dapat menghasilkan perubahan tegangan berdasarkan posisi putarannya. Pada mini project ini, potensiometer digunakan sebagai input analog untuk mengatur parameter sistem.

Potensiometer kecepatan memetakan nilai ADC ke rentang delay perpindahan LED. Potensiometer brightness memetakan nilai ADC ke rentang duty cycle PWM, sehingga tingkat kecerahan LED dapat diatur secara manual.

## **2.5 WiFi Scanner Asinkron pada ESP32**

ESP32 memiliki fitur WiFi yang dapat digunakan untuk melakukan pemindaian jaringan di sekitar perangkat. Fungsi `WiFi.scanNetworks(true)` memungkinkan proses pemindaian dilakukan secara asinkron atau non-blocking.

Dengan mode asinkron, ESP32 tetap dapat menjalankan program utama seperti animasi LED dan pembacaan sensor saat proses scanning WiFi sedang berlangsung. Hal ini penting karena proses scanning WiFi dapat memerlukan waktu beberapa detik jika dilakukan secara blocking.

### **3 Implementasi: Lampu Berjalan, Kontrol LDR, Dimming, dan WiFi Scanner**

#### **3.1 Deskripsi**

Mini project ini mengimplementasikan efek lampu berjalan atau Knight Rider pada 3 buah LED. LED menyala secara bergantian dari kiri ke kanan dan kembali lagi secara berulang. Sistem dilengkapi dengan dua buah potensiometer, yaitu potensiometer untuk mengatur kecepatan animasi dan potensiometer untuk mengatur kecerahan LED.

Selain itu, sistem menggunakan sensor LDR untuk mendeteksi kondisi cahaya lingkungan. Jika kondisi lingkungan dianggap gelap, maka efek lampu berjalan akan aktif. Sebaliknya, jika kondisi lingkungan terang, seluruh LED akan dimatikan. ESP32 juga melakukan pemindaian jaringan WiFi secara berkala dan menampilkan hasilnya pada Serial Monitor.

#### **3.2 Komponen yang Digunakan**

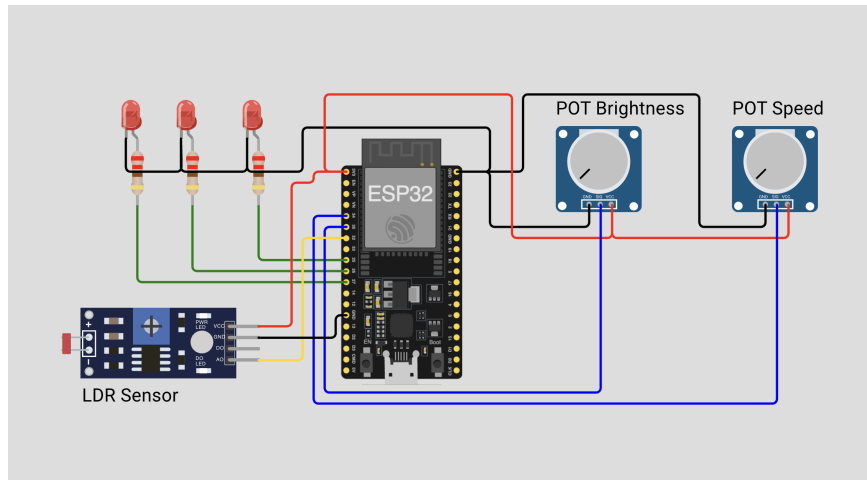
Komponen yang digunakan pada mini project ini adalah sebagai berikut:

- Mikrokontroler ESP32.
- 3 buah LED.
- 3 buah resistor  $220\Omega$ .
- 2 buah potensiometer  $10k\Omega$ .
- 1 buah modul sensor LDR.
- Kabel jumper.
- Breadboard.

#### **3.3 Rangkaian (Wiring Diagram)**

Rangkaian terdiri dari tiga LED yang masing-masing terhubung ke pin output digital ESP32. Dua potensiometer dihubungkan ke pin ADC untuk mengatur kecepatan dan brightness. Sensor LDR dihubungkan dalam

konfigurasi pembagi tegangan agar perubahan intensitas cahaya dapat dibaca sebagai perubahan nilai ADC.



**Gambar 1:** Wiring diagram lampu berjalan dengan LDR dan potensiometer

Tabel 1 berikut menjelaskan detail koneksi pin antara ESP32 dengan komponen-komponen pada rangkaian lampu berjalan.

**Tabel 1:** Detail Koneksi Pin Rangkaian Lampu Berjalan

| Komponen                | Pin Komponen                       | Pin ESP32                   |
|-------------------------|------------------------------------|-----------------------------|
| LED 1                   | Anode (+)<br>Cathode (–)           | GPIO 25 (via R 220Ω)<br>GND |
| LED 2                   | Anode (+)<br>Cathode (–)           | GPIO 26 (via R 220Ω)<br>GND |
| LED 3                   | Anode (+)<br>Cathode (–)           | GPIO 27 (via R 220Ω)<br>GND |
| Potensiometer Kecepatan | Kaki 1<br>Wiper (tengah)<br>Kaki 3 | 3V3<br>GPIO 34<br>GND       |
| Potensiometer Kecerahan | Kaki 1<br>Wiper (tengah)<br>Kaki 3 | 3V3<br>GPIO 35<br>GND       |
| Sensor LDR              | VCC<br>GND<br>A0                   | 3V3<br>GND<br>GPIO 32       |

### 3.4 Kode Program

Kode program berikut digunakan untuk menjalankan efek lampu berjalan, membaca nilai LDR, membaca dua potensiometer, mengatur brightness LED

menggunakan PWM, serta melakukan scanning WiFi secara asinkron.

```
1 /**
2  * @author Yuke Brilliant Hestiavin 5024241016
3  * Knight Rider Running Light with Brightness Control and
4  * Speed Control using Potentiometer,
5  * LDR control, and async WiFi Network Scanner
6  */
7
8 #include <Arduino.h>
9 #include <WiFi.h>
10
11 // LED Pins
12 #define LED1_PIN 25
13 #define LED2_PIN 26
14 #define LED3_PIN 27
15
16 // Potentiometer Pins
17 #define POT_PIN_SPEED 34
18 #define POT_PIN_BRIGHTNESS 35
19
20 // LDR Pin
21 #define LDR_PIN 32
22 const int LDR_THRESHOLD = 2000;
23
24 const unsigned long WIFI_SCAN_INTERVAL = 10000;
25 unsigned long lastScanTime = 0;
26 bool scanRunning = false;
27
28 const int ledSequence[] = {LED1_PIN, LED2_PIN, LED3_PIN, LED2_PIN};
29 const int sequenceLength = 4;
30
31 int currentLedIndex = 0;
32 unsigned long lastLedMoveTime = 0;
33
34 void turnOffAllLEDs() {
35     analogWrite(LED1_PIN, 0);
36     analogWrite(LED2_PIN, 0);
37     analogWrite(LED3_PIN, 0);
38 }
39
40 void runKnightRider() {
41     int ldrValue = analogRead(LDR_PIN);
42
43     Serial.print("LDR Value: ");
44     Serial.print(ldrValue);
45
46     if (ldrValue > LDR_THRESHOLD) {
47         Serial.println(" - It's dark, running Knight Rider effect.");
48     }
49 }
```

```

48
49     int speedValue = analogRead(POT_PIN_SPEED);
50     int brightnessValue = analogRead(POT_PIN_BRIGHTNESS);
51
52     int speedDelay = map(speedValue, 0, 4095, 50, 500);
53
54     int brightness = map(brightnessValue, 0, 4095, 0, 255);
55
56     if (millis() - lastLedMoveTime >= speedDelay) {
57         lastLedMoveTime = millis();
58
59         turnOffAllLEDs();
60         analogWrite(ledSequence[currentLedIndex], brightness);
61
62         currentLedIndex++;
63         if (currentLedIndex >= sequenceLength) {
64             currentLedIndex = 0;
65         }
66
67         Serial.print("Speed Delay: ");
68         Serial.print(speedDelay);
69         Serial.print(" ms, Brightness: ");
70         Serial.println(brightness);
71     }
72
73     } else {
74         Serial.println(" - It's bright, turning off LEDs.");
75         turnOffAllLEDs();
76     }
77 }
78
79 void handleWiFiScan() {
80     int scanResult = WiFi.scanComplete();
81
82     if (scanRunning && scanResult >= 0) {
83         Serial.printf("\nFound %d networks:\n", scanResult);
84
85         if (scanResult == 0) {
86             Serial.println("No networks found.");
87         } else {
88             Serial.println("Network List:");
89             for (int i = 0; i < scanResult; ++i) {
90                 Serial.printf(
91                     "%d: SSID: %s, RSSI: %d dBm, Channel: %d, Encryption: %s\n",
92                     i + 1,
93                     WiFi.SSID(i).c_str(),
94                     WiFi.RSSI(i),
95                     WiFi.channel(i),

```



```

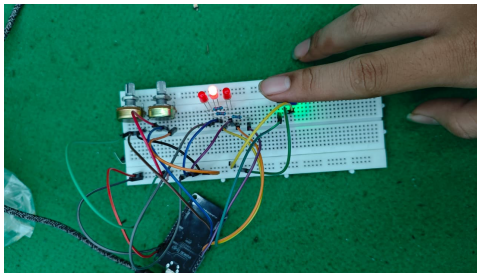
96         (WiFi.encryptionType(i) == WIFI_AUTH_OPEN) ? "Open" : "Secured"
97     );
98 }
99 }
100
101 Serial.println("=====");
102
103 WiFi.scanDelete();
104 scanRunning = false;
105 lastScanTime = millis();
106 }
107
108 if (!scanRunning && millis() - lastScanTime >= WIFI_SCAN_INTERVAL) {
109     Serial.println("\n=== WiFi Network Scanner ===");
110     Serial.println("Scanning for WiFi networks...");
111     Serial.println("=====");
112
113     WiFi.scanNetworks(true); // true = async scan
114     scanRunning = true;
115 }
116 }
117
118 void setup() {
119     Serial.begin(115200);
120
121     pinMode(LED1_PIN, OUTPUT);
122     pinMode(LED2_PIN, OUTPUT);
123     pinMode(LED3_PIN, OUTPUT);
124
125     pinMode(POT_PIN_SPEED, INPUT);
126     pinMode(POT_PIN_BRIGHTNESS, INPUT);
127     pinMode(LDR_PIN, INPUT);
128
129     WiFi.mode(WIFI_STA);
130     WiFi.disconnect();
131
132     Serial.println("System started.");
133
134     lastScanTime = millis() - WIFI_SCAN_INTERVAL;
135 }
136
137 void loop() {
138     handleWiFiScan();
139     runKnightRider();
140
141     delay(50);

```

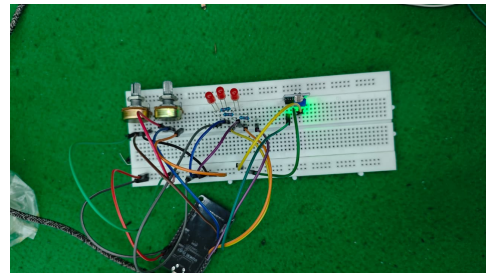
**Listing 1:** Kode program lampu berjalan, kontrol LDR, dimming, dan WiFi scanner

### 3.5 Hasil dan Pembahasan

Sistem berhasil mengimplementasikan semua fitur yang direncanakan. Sensor LDR membaca nilai intensitas cahaya lingkungan melalui pin ADC ESP32. Jika nilai ADC LDR melebihi nilai threshold 2000, sistem menganggap kondisi lingkungan sedang gelap dan mengaktifkan efek lampu berjalan. Sebaliknya, jika nilai ADC berada di bawah threshold, sistem menganggap kondisi lingkungan terang dan seluruh LED dimatikan.



**Gambar 2:** Kondisi lampu berjalan menyala saat lingkungan gelap



**Gambar 3:** Kondisi lampu berjalan mati saat lingkungan terang

Nilai dari kedua potensiometer dibaca secara terpisah oleh ESP32. Potensiometer pertama digunakan untuk mengatur kecepatan animasi dengan memetakan nilai ADC 0 hingga 4095 ke rentang delay 50 hingga 500 ms. Jika nilai delay semakin kecil, maka perpindahan LED akan semakin cepat. Jika nilai delay semakin besar, maka perpindahan LED akan semakin lambat.

Potensiometer kedua digunakan untuk mengatur brightness LED. Nilai ADC dari potensiometer dipetakan ke rentang duty cycle PWM 0 hingga 255. Duty cycle yang lebih besar menghasilkan LED yang lebih terang, sedangkan duty cycle yang lebih kecil menghasilkan LED yang lebih redup.

```

Speed Pot Value: 1885, Brightness Pot Value: 811
LDR Value: 3478
Speed Pot Value: 1883, Brightness Pot Value: 1085
LDR Value: 3478
Speed Pot Value: 1894, Brightness Pot Value: 1364
LDR Value: 3491
Speed Pot Value: 1886, Brightness Pot Value: 1665
LDR Value: 3504
Speed Pot Value: 1883, Brightness Pot Value: 1916
LDR Value: 3511
Speed Pot Value: 1883, Brightness Pot Value: 2189
Speed Delay: 256 ms, Brightness: 136
LDR Value: 3506
Speed Pot Value: 1883, Brightness Pot Value: 2493
LDR Value: 3495
Speed Pot Value: 1886, Brightness Pot Value: 2863
LDR Value: 3491
Speed Pot Value: 1873, Brightness Pot Value: 3355
LDR Value: 3483
Speed Pot Value: 1873, Brightness Pot Value: 3984
LDR Value: 3493
Speed Pot Value: 1884, Brightness Pot Value: 4095
LDR Value: 3503
Speed Pot Value: 1882, Brightness Pot Value: 4095
Speed Delay: 256 ms, Brightness: 255
LDR Value: 3531
Speed Pot Value: 1891, Brightness Pot Value: 4095
LDR Value: 3536
Speed Pot Value: 1895, Brightness Pot Value: 4095
LDR Value: 3557
Speed Pot Value: 1891, Brightness Pot Value: 4095
LDR Value: 3583
Speed Pot Value: 1894, Brightness Pot Value: 4095
LDR Value: 3583
Speed Pot Value: 1893, Brightness Pot Value: 4095
LDR Value: 3583
Speed Pot Value: 1895, Brightness Pot Value: 4095

```

**Gambar 4:** Output Serial Monitor pembacaan potensiometer dan LDR

Pada Gambar 4, Serial Monitor menampilkan nilai pembacaan dari potensiometer dan LDR. Data tersebut digunakan untuk memantau apakah input analog terbaca dengan benar oleh ESP32. Tampilan Serial Monitor juga membantu dalam proses debugging karena perubahan nilai potensiometer dapat langsung diamati.

Fitur WiFi scanner berjalan secara asinkron setiap interval tertentu. Penggunaan `WiFi.scanNetworks(true)` membuat proses pemindaian jaringan WiFi tidak memblokir eksekusi program utama. Dengan demikian, animasi lampu berjalan tetap dapat berjalan, pembacaan sensor tetap dilakukan, dan sistem tidak berhenti sementara saat proses scanning WiFi berlangsung.

Hasil pemindaian WiFi ditampilkan pada Serial Monitor, meliputi nama jaringan atau SSID, kekuatan sinyal atau RSSI, channel, dan status enkripsi. Hal ini menunjukkan bahwa ESP32 tidak hanya dapat digunakan untuk membaca sensor dan mengontrol output, tetapi juga dapat memanfaatkan konektivitas nirkabel untuk kebutuhan monitoring atau komunikasi data.

## 4 Kesimpulan

Berdasarkan mini project yang telah dibuat, dapat disimpulkan bahwa:

1. PWM pada ESP32 dapat digunakan untuk mengatur kecerahan LED secara dinamis berdasarkan input analog dari potensiometer.
2. ADC pada ESP32 mampu membaca beberapa input analog sekaligus, yaitu potensiometer kecepatan, potensiometer brightness, dan sensor LDR.

```

Found 7 networks:
Network List:
1: SSID: ROBOTIKA, RSSI: -60 dBm, Channel: 6, Encryption: Secured
2: SSID: ICHIRO KRI 1ST, RSSI: -63 dBm, Channel: 3, Encryption: Secured
3: SSID: Robotika@test, RSSI: -64 dBm, Channel: 11, Encryption: Secured
4: SSID: Robotika@test, RSSI: -79 dBm, Channel: 1, Encryption: Secured
5: SSID: Robotika@test, RSSI: -87 dBm, Channel: 6, Encryption: Secured
6: SSID: waiiiii, RSSI: -92 dBm, Channel: 11, Encryption: Secured
7: SSID: ROBOTAN, RSSI: -93 dBm, Channel: 6, Encryption: Secured

Found 6 networks:
Network List:
1: SSID: ROBOTIKA, RSSI: -59 dBm, Channel: 6, Encryption: Secured
2: SSID: Robotika@test, RSSI: -63 dBm, Channel: 11, Encryption: Secured
3: SSID: ICHIRO KRI 1ST, RSSI: -64 dBm, Channel: 3, Encryption: Secured
4: SSID: waiiiii, RSSI: -85 dBm, Channel: 11, Encryption: Secured
5: SSID: Robotika@test, RSSI: -89 dBm, Channel: 6, Encryption: Secured
6: SSID: ROBOTAN, RSSI: -94 dBm, Channel: 6, Encryption: Secured

```

**Gambar 5:** Output Serial Monitor hasil pemindaian jaringan WiFi

3. Sensor LDR dapat digunakan sebagai input otomatis untuk mengaktifkan atau menonaktifkan sistem berdasarkan kondisi cahaya lingkungan.
4. Potensiometer dapat digunakan sebagai kontrol manual yang sederhana untuk mengatur parameter sistem seperti kecepatan animasi dan tingkat kecerahan LED.
5. Fitur WiFi scanning asinkron pada ESP32 memungkinkan proses pemindaian jaringan berjalan tanpa mengganggu tugas utama sistem.
6. Mini project ini berhasil menggabungkan konsep PWM, ADC, sensor, aktuator, dan konektivitas WiFi dalam satu sistem embedded sederhana.